
Who Grades the Grader? The Verification Gap in Self-Testing Code Agents

Brian Sam-Bodden
Integrallis Software

Jyoti Vasudev
Microsoft

Josh Meyers
Independent Researcher

Eric Kasper
Independent Researcher

Abstract

When a coding agent writes both code and tests, the two can agree while sharing the same mistake. Passing those tests may then conceal a misunderstood requirement. We measure how often code reported as successful fails tests that were not provided to the agent. We call this mismatch the *verification gap*.

We evaluate one test-first pipeline on selected HumanEval and LiveCodeBench problems and on RGRBench, a collection of 89 Python packages. With one inexpensive model writing tests, generating code, and reviewing both, the withheld tests reject 18.5% of claimed successes on HumanEval, 28.9% on LiveCodeBench, and 41.5% on RGRBench.

We then test stronger models as test authors and reviewers while keeping the code generator fixed. Across three providers on the single-function benchmarks, some settings produce fewer success claims that fail the withheld tests, but the evidence for a reduction remains inconclusive. In our main LiveCodeBench experiment, a stronger reviewer also helps the pipeline produce more solutions that pass the withheld tests. On RGRBench, the same reviewer stops many tasks before code is written and produces fewer passing solutions. Failures remain among claimed successes in every configuration studied.

The practical lesson is to measure both what an agent produces and how reliably it judges its own work. That judgment also depends on a clear task: some benchmark failures reflect requirements that omit details demanded by the tests. We release the pipeline, recorded runs, and analysis tools to support independent checks of coding agents' success claims.

1 Introduction

Test-Driven Development (TDD) [3] uses a Red–Green–Refactor loop: write a failing test for a requirement, implement behavior that passes it, then improve the implementation while preserving the tests. Its usefulness depends on the tests representing the intended behavior. Passing an executable contract cannot establish that the contract itself captures the requirement.

Agent pipelines now run that loop independently, writing their own acceptance criteria. An LLM takes a natural-language specification and turns it into a test suite, a second generation writes code to pass those tests, LLM-as-a-judge reviewers check each step, and the pipeline declares success when its code passes its tests. The pipeline comes with its own evidence.

But TDD can still fall victim to misunderstandings of the requirements. Those misinterpretations are now locked into code that drives the pipeline. What is the value of passing tests if they are checking the wrong thing? TDD practitioners already have a term for tests that mirror the code instead of the requirement: the *ugly mirror* [16].

We will show that autonomous test-first pipelines create the conditions for the ugly mirror at the stage where natural language is turned into tests. We call the resulting mismatch the *verification*

gap: problems that a pipeline claims to have solved but that fail when checked by an oracle the pipeline never sees (Sec. 3).

This paper measures that gap by stress-testing an agentic test-first pipeline. In the control setup, using a low-cost, non-frontier model in every role (hereafter *all-weak*), the gap is persistent and appears on both sides. Out of 30 problems per run, an average of 4.0 reported successes fail HumanEval’s strong oracle (18.5% of what the pipeline claims) and 4.8 fail the hidden oracle of a LiveCodeBench window that post-dates the generator’s training cutoff (29% of claims), while 5.0 and 3.4 oracle-passing implementations per run are rejected by the pipeline’s tests or internal gates.

The most common fix for a model that misunderstands a requirement is to use a stronger model to write the tests. LLM-as-a-judge is also widely used to review the generated code at every stage. Production systems are increasingly using this asymmetric approach: a cheap model generates, and a frontier model is used somewhere in the verification roles. The *test author* turns the specification into a test suite, and the *judge* reviews the tests and code and examines the failures. We test these approaches as well: we place a frontier model in the test-author role, the judge role, and both, while keeping a fixed weak generator (gpt-4o-mini) on both benchmarks. We then repeat the design across three frontier verifiers and three providers (GPT-5.6 Terra, Gemini-2.5-Pro, Claude-4.5-Sonnet; Sec. 4). The primary experiment suggests an asymmetry: the frontier judge improves oracle success on LiveCodeBench, while residual false accepts remain in all conditions. No false-accept contrast passes the specified decision bar, but this is a failure to establish a reduction, not evidence that verifier strength cannot help. Some false-accept point estimates decrease substantially. Counts must also be separated from the error rate among accepted outputs because the interventions change acceptance and whether the pipeline produces code at all.

From algorithm to application scale. Both HumanEval and LiveCodeBench above were designed as single-function, mostly algorithmic problems and do not represent software applications that have stateful behavior, multiple files, classes, or components, and complex collaboration patterns between them. To examine the gap on larger tasks, we built RGRBench: 89 Python packages spanning small kata and larger stateful, multi-concern examples (event sourcing, reservations, billing) across three difficulty levels, each with a hand-reviewed suite assessed using mutation testing (mutation kill rate 0.981, with surviving mutants classified). This set of applications was created in a TDD style by translating and curating TDD examples from other programming languages when no Python version was available.

Our pipeline receives each package’s extracted requirements, with benchmark code and tests withheld. Across the 534 planned runs, the control’s conditional gap is 41.5% and the strong judge’s is 42.0%. Higher observed gap rates accompany harder packages and additional GREEN attempts. These comparisons do not isolate causal effects of complexity or retries. The stronger judge also blocks many more packages before code generation, reducing total correct output. RGRBench extends the measurement to larger tasks while exposing selection and oracle-fidelity limitations that must accompany its headline rate.

Contributions. (1) An execution-grounded protocol for comparing a self-testing pipeline’s verdict with external oracles, distinguishing false accepts, false rejects, and conditional error rates (Sec. 3). (2) Role-assignment experiments on selected HumanEval and LiveCodeBench problems, replicated with three frontier verifiers, reporting residual errors and interventions that improve oracle success (Sec. 5.2). (3) An application-scale study on RGRBench with a full confusion matrix and a threshold follow-up, exposing the tradeoff between acceptance and correct output (Sec. 5.4). (4) Released runners, retained results, analysis code, prompt templates, and a defect register, with disclosures of protocol deviations and the limits of the reconstructed public history.¹

2 Background and Related Work

Weak tests overestimate, in benchmarks and pipelines alike. EvalPlus showed that the tests in the benchmark were the weak point in code evaluation: by expanding HumanEval’s test suites, it found a significant amount of previously unnoticed incorrect code and changed the order of model

¹Artifact: <https://github.com/integrallis/self-grading-gap>. RGRBench: <https://github.com/integrallis/rgrbench>.

rankings [13]. LiveCodeBench addressed the related issue of contamination by using a rolling problem collection approach [10]. We used both tools. EvalPlus serves as a strengthened oracle, and for LiveCodeBench, we rely on a post-cutoff window as a hidden oracle. In the TDD pipeline, when the tests are weak, this overestimation happens inside the loop, and the pipeline’s own self-assessment shows the same lack of awareness that EvalPlus found in benchmarks. At repository scale, Ahmed et al. compare generated-test-passing patches with held-out golden tests [1], while SWE-ABS re-evaluates benchmark-passing patches with strengthened suites [20]. What we call the verification gap is the TDD-internal, agent-native version of this phenomenon, measured with additional mutation and adapter checks on RGRBench. These checks characterize the instrument; they do not estimate its population error rate. The cited SWE-bench studies use different acceptance events and denominators, so their rates are contextual comparisons.

The ugly mirror. TDD practitioners have long described the failure that happens when tests are written after the implementation already exists: the *ugly mirror* [16], where a test simply reflects the code and confirms what it does, rather than what it should do, with the tautological test as the extreme case [15]. Writing tests before the code removes direct dependence on an existing implementation, but cannot prevent misunderstanding the specification. Our subject pipelines use this order, and yet we see that the mirror effect still appears, just at a higher level. When a model writes tests based on its interpretation of the specification and a similar model writes code to pass those tests, both the tests and the code can reflect the same misunderstanding, and successful test execution then establishes agreement on the generated cases. The pipeline then reports this internal consistency as success, but it only shows that the tests and the code are in agreement.

Test-first generation with LLMs. Providing tests with problem statements improves generation [14]; TiCoder makes the authorship of tests more interactive—a human checks the proposed tests before the code is accepted [6]; TENET and TDFlow extend test-driven loops to repository-scale agents [9, 7]. Testing is provided, verified by humans, or evaluated end-to-end by the impact on pass rates. Our subject class is the fully autonomous case, in which the pipeline authors both tests and code. What we are interested in is the reliability of the pipeline’s own verdict on itself. End-to-end pass rates alone do not reveal whether a pipeline’s internal acceptance correctly distinguishes the outputs that will pass the held-out oracle.

Self-generated tests as a signal. CodeT uses generated tests to rank candidate solutions through execution agreement [5]. ConVerTest combines self-consistency, code verification, and execution agreement, evaluating test validity and coverage [19]. Such signals can improve selection without guaranteeing correctness. Our study measures residual oracle disagreement in a different, test-first pipeline; it does not reproduce or refute those methods’ reported improvements.

LLM judges on code. Benchmarks evaluating LLMs as judges for coding tasks show that these judges are imperfectly aligned with execution results, with performance varying by model and task [11], and that LLMs consistently struggle when checking code against natural-language specifications [12]. In our subject pipeline, we include judges as checkpoints in the process, along with an execution gate that can override the judge’s predictions. The asymmetry experiments extend these findings: upgrading the judge to a frontier model is associated with higher oracle success in the primary LiveCodeBench experiment, consistent with useful feedback, while some accepted outputs still fail the oracle. Because changing the judge also changes generation, this experiment does not isolate its classification accuracy on a fixed set of implementations.

Multi-agent code generation. Multi-agent SDLC frameworks have been effective at handling end-to-end tasks [8]. End-to-end performance is complementary to auditing internal acceptance gates. Our protocol offers a complementary and transferable approach: any pipeline with a harness that records its executed self-verdict and outputs can use the same measurement.

The oracle problem and imperfect verification. The difficulty of deciding whether software behaves correctly is the established oracle problem in software testing [2]. Work on inference scaling analyzes imperfect verifiers when selecting among repeated code-generation attempts [18]. Our contribution is a measurement of this problem inside an executed test-first pipeline, with role interventions and accounting of accepted and rejected outputs; the general limitation of incomplete tests is not new.

Connection to context ablations. Sam-Bodden [17] studies what code representation a coding agent needs after localization is fixed, using independent SWE-bench outcomes and repeated API runs. That study motivates the same separation between an agent’s intermediate representation and external evidence of task success. Here the intermediate artifact is a self-authored test suite. The benchmarks, interventions, and outcome definitions differ, so the context study is complementary methodological work rather than a replication of the verification gap.

3 The Verification Gap

3.1 Definitions

Consider a code-generation pipeline that, for a problem i with natural-language specification $spec_i$, produces an implementation y_i , a test suite T_i that the pipeline itself wrote from $spec_i$, and a self-verdict $s_i \in \{0, 1\}$, the pipeline’s own claim that the problem is solved, based on running T_i against y_i and from any internal quality gates (in our subject pipeline, LLM judges). An *oracle verdict* $o_i \in \{0, 1\}$ is the outcome of a test suite the pipeline never sees. We use three oracles of increasing distance from the pipeline’s inputs: the *visible-adjacent oracle* o_i^{base} (HumanEval’s original tests, which overlap the worked examples visible in $spec_i$), a *strong oracle* o_i^{plus} (HumanEval+ [13], whose tests go far beyond the visible examples, by roughly two orders of magnitude), and a *hidden oracle* (LiveCodeBench’s private test sets [10], never distributed with the problem statement at all).

The *verification gap* of a run over problems P is the set $G = \{i \in P : s_i = 1 \wedge o_i = 0\}$: problems the pipeline believes it solved and the oracle rejects, the *false accepts*. We report it as a per-run count over $|P|$ and, where the denominator matters, as the *conditional gap* $|G|/|\{i : s_i = 1\}|$, the observed fraction of claims rejected by the chosen oracle. The complementary error set $R = \{i : s_i = 0 \wedge o_i = 1\}$ (*false rejects*: oracle-passing code rejected by the pipeline’s tests or internal gates) completes the picture; self-verification error runs in both directions (Table 1).

Table 1: Self-verification as a 2×2 confusion matrix: the pipeline’s self-verdict s_i against the held-out oracle o_i . The loop observes only the rows—its own verdict; the columns are hidden from it. Both off-diagonal cells are errors, but they differ in visibility: a *false reject* is rejected by the pipeline and can be re-examined, whereas a *false accept* receives the same positive self-verdict as a true success. The verification gap G is the upper-right cell (what *soundness* concerns); the false rejects R are the lower-left (what *completeness* concerns).

	oracle pass ($o_i=1$)	oracle fail ($o_i=0$)
self-verdict accept ($s_i=1$)	true accept	false accept (G)
self-verdict reject ($s_i=0$)	false reject (R)	true reject

Every element of G is an observed disagreement: an accepted implementation passes the pipeline’s execution gate but fails an external suite. An incomplete self-suite, an incorrect expected value, an implementation defect, an underspecified requirement, or an oracle or adapter error can all produce this observation. The definition alone does not establish that the implementation violates the visible specification or that tests and code share a misunderstanding. We use *self-consistent misalignment* for cases where inspection supports that interpretation, and retain *oracle-relative false accept* for the measured event. HumanEval case studies illustrate missed specified behavior; the RGRBench example in Appendix D instead exposes a requirements–oracle mismatch.

3.2 Subject pipeline

Our subject is a test-first multi-agent pipeline in which an LLM writes tests from the specification (RED), an LLM judge scores the tests, a second generation step writes an implementation against those tests (GREEN), a judge predicts test outcomes, an execution gate then runs the tests and overrides optimistic judge predictions, and a final VERIFY phase runs the full suite, with bounded retries and textual feedback loops (Algorithm 1, Fig. 1). The pipeline’s main configuration also *anchors* self-authored tests: prompt and judge together require that every worked example in the specification appear as a test case exactly as given. Note what the execution gate cannot do: it checks y against T , so a wrong T passes it incorrectly.

Algorithm 1 Execution-gated TDD cycle (subject pipeline; generator G and judge J are role-assigned per experimental cell)

Require: spec, test author G_T , code generator G , judge J , threshold τ , retries R

```

1:  $f \leftarrow \emptyset$ 
2: for  $r \leftarrow 1 \dots R$  do ▷ RED: author the tests
3:    $T \leftarrow G_T.\text{tests}(\text{spec}, f)$ ;  $q \leftarrow J.\text{score}(T, \text{spec})$ ;  $f \leftarrow J.\text{feedback}$ 
4:   if  $q \geq \tau$  then break
5:   end if
6: end for
7: if  $q < \tau$  then return  $s=0$ 
8: end if
9:  $g \leftarrow 0$ 
10: for  $r \leftarrow 1 \dots R$  do ▷ GREEN: code against own tests
11:    $y \leftarrow G.\text{code}(\text{spec}, T, f)$ ;  $\hat{q} \leftarrow J.\text{predict}(y, T)$ 
12:   if  $\hat{q} \geq \tau$  then
13:     if  $\text{run}(T, y).\text{allPass}$  then  $g \leftarrow 1$ ; break ▷ execution gate: judge overruled by pytest
14:     else  $f \leftarrow \text{debugAnalysis}(T, y)$ 
15:     end if
16:   else  $f \leftarrow J.\text{feedback}$ 
17:   end if
18: end for
19: if  $g = 0$  then return  $s=0, y, T$ 
20: end if
21: for  $r \leftarrow 1 \dots R$  do ▷ VERIFY: full suite; failures route back to GREEN
22:   if  $\text{run}(T, y).\text{allPass}$  then return  $s=1, y, T$ 
23:   end if
24:    $f \leftarrow J.\text{analyzeFailures}(T, y)$ ; regenerate  $y$  as in GREEN
25: end for
26: return  $s=0, y, T$ 

```

The complete instrument (engine, prompt templates, and the defect register from a line-by-line review of the pipeline) is included with the artifact.

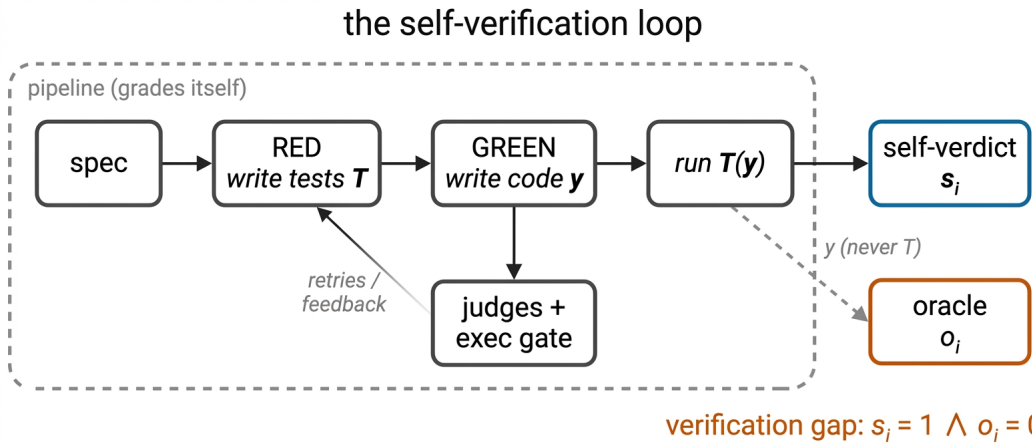


Figure 1: The pipeline authors tests T and code y , grades itself by running $T(y)$, and emits self-verdict s_i ; the oracle sees y but never T . The verification gap is the set of problems where the two verdicts disagree in the pipeline’s favor.

Algorithm 2 Verification-gap measurement over instrumented runs

Require: pipeline P , problems $\mathcal{P}$, held-out oracle suites O , runs N

```
1: for each problem  $i \in \mathcal{P}$ , run  $r \in 1..N$  do  
2:    $(y_{ir}, T_{ir}, s_{ir}) \leftarrow P(i)$   $\triangleright$  harness captures artifacts + the executed self-verdict  
3:    $o_{ir} \leftarrow \text{Exec}(O_i \text{ on } y_{ir})$   $\triangleright$  standard harness (EvalPlus / LCB / RGRBench adapter), outside  
   the loop  
4: end for  
5:  $G \leftarrow \{(i, r) : s_{ir}=1 \wedge o_{ir}=0\}$ ;  $R \leftarrow \{(i, r) : s_{ir}=0 \wedge o_{ir}=1\}$   
6: return  $G, R$ , per-run provenance and cost
```

3.3 Prompt architecture

Four prompts drive the loop; their structure is part of the instrument and matters to the results, so we describe it. (Skeletons in Appendix B; the templates come with the artifact under a SHA-256 manifest, with render-contract tests asserting the properties below.)

RED generator. Role framing (“your tests are the only specification the implementer will see”), task description and full specification, scenario list, and finally an *anchoring block*: for every worked example in the specification, a test named `test_canonical_example_N` with the example’s exact inputs and outputs and a one-line derivation comment. There are five rules that must be followed: only test the specified behavior, use exact signatures, compute expected values, import from the solution package, and use plain pytest. On retries, the judge’s directives and the previous attempt are included. The output contract is a single fenced Python block.

RED judge. The judge considers a single question: does the suite faithfully verify the specification? This is done through a series of ordered checks: canonical-example coverage (an anchoring block that mirrors the generator’s; any missing or changed example results in the minimum score), assertion fidelity (recalculate each expected value), no invented constraints, coverage of all stated behaviors, and sanity. A deduction schedule links findings to a score; the response always ends with the line `Final score: <x.xx>`, and feedback is given as `apply-verbatim` `ADD` `:REMOVE` `:FIX` directives, which is the feedback channel examined in Sec. 6.

GREEN generator. The tests are given as the acceptance criterion; the rules prohibit hardcoding test inputs and inventing behavior; the output is a single JSON object containing the implementation file (with path, action, and content). This is the only structured output where code appears as an escaped string field.

GREEN judge. Predicts per-test PASS/FAIL by tracing the implementation against each test, flags test-gaming in one line and must end with `Rating: <x.xx>` (predicted pass fraction). If the judge clears the threshold, the execution gate runs the tests, so a positive judge prediction alone cannot produce an accepted output.

The anchoring blocks in the RED pair are delimited by markers in the template source, and the ablation variants of the artifact (generator only, judge-side only, neither) are produced by stripping those blocks automatically; the variant used in every run is checked on the rendered prompt at smoke time. A fifth template, the failure analyst of the retry loops in Sec. 3, is triggered when an execution gate or VERIFY run fails. It follows each failed test and identifies the root cause and emits `FIX DIRECTIVES`: lines that the next GREEN attempt uses verbatim; it presents analysis only, never code.

3.4 Measurement protocol

Three requirements make the gap measurable rather than anecdotal (Algorithm 2).

(1) *Both verdicts captured independently.* The self-verdict s_{ir} is the pipeline graph’s recorded success flag. Acceptance requires successful final test execution; rejection can instead arise at a judge gate or retry limit, including before an implementation exists. It is not simply a binary result of executing the final self-suite. Runs retain per-call token and cost accounting. The oracle verdict is obtained on the produced implementation outside the loop by the benchmark’s own harness: EvalPlus [13], LiveCodeBench’s evaluator [10] for the hidden-test problems, and RGRBench’s hand-verified suites through a non-cheating adapter (Sec. 5.4). Every number in this paper comes from

these two recorded streams, the self-verdict and the oracle, and can be rebuilt from committed per-run artifacts.

(2) *Oracles beyond the visible examples.* Anchoring ties T_i to the specification’s worked examples, which HumanEval’s base oracle partially shares; a gap measured only against o^{base} understates the phenomenon. On the all-weak setup, false accepts average 3.0 per 30-problem run against the base oracle and 4.0 against HumanEval+. The LiveCodeBench experiment removes visibility entirely: prompts include the statement and its public examples, while scoring uses the private suites, on problems whose contest dates are more than a year after the weak model’s training cutoff.

(3) *Repeated-run variability and practical thresholds.* Per-problem outcomes are unstable across byte-identical runs: the all-weak setup disagrees with itself on 6/30 problems (HumanEval subset) and 9/30 (LiveCodeBench) across five runs, with maximum pairwise pass-total differences of 4 and 3. The protocols specify minimum effect sizes (4 problems on success-rate contrasts, 2 on false accepts, 2 on false rejects, 3 on total verdict error) alongside statistical significance. These practical thresholds are not confidence bounds or equivalence margins; some control error-count spreads exceed them. The observed variability cautions against treating small single-run differences as stable effects.

4 Study Design

The study asks four questions.

RQ1 (phenomenon). How large is the verification gap, against visible-adjacent, strong, and hidden oracles, and how does it break down into false accepts and false rejects?

RQ2 (asymmetry). Does a frontier model in the verification roles (as test author, as LLM-as-a-judge reviewer, or both) close the gap?

RQ3 (mechanism). When a stronger verifier does help, does the improvement come from its *verdicts* (better gatekeeping) or from its *feedback* (better generation)?

RQ4 (scale). Do the size of the gap and its resistance to verifier strength persist when moving from single-function algorithms to larger, application-scale programs that involve multiple concerns, and how does this gap change with increasing task complexity or as the loop itself is refined?

Experiments. The model experiments use a single asymmetry design with four conditions (*cells*): a weak model W (gpt-4o-mini) generates code in every cell; a frontier verifier S replaces W in the test-author role, the judge role (covering all three judge duties), or both; the control condition uses W in every role. We assign the S role to three verifiers from three different providers—GPT-5.6 Terra, Gemini-2.5-Pro, and Claude-4.5-Sonnet²—and run this design on both benchmarks for each verifier. $N=5$ runs per cell, 30 problems per run, model identity is fixed within each assigned role, and judge threshold and retries are held constant. The single-function runners can switch late generation attempts to a second client for the same model at a different temperature. The protocols record advance hypotheses, but the public release history was reconstructed: original chronology depends on private commits (PROVENANCE.md). Exp011 documents collection and retention changes during the runs; these limit a strictly confirmatory interpretation. The primary pair uses GPT-5.6 Terra: exp006 runs a HumanEval subset that includes previously unstable problems, scored by both EvalPlus oracles, and exp007 repeats this on contamination-controlled data—all 19 easy plus the first 11 medium LeetCode problems from LiveCodeBench release v6 with contest dates $\geq 2025-01-01$, chosen by a deterministic rule fixed in advance after a pilot identified the weak model’s measurable range, and scored by the private LCB suites. The exp007 protocol records predictions of fewer false rejects and no detectable false-accept contrast, following exp006. The exp011 replication then reruns both benchmark designs with the two non-OpenAI verifiers, Gemini-2.5-Pro and Claude-4.5-Sonnet, to test whether the soundness null depends on which frontier model is used (Sec. 5.2).

The application-scale experiment. A third experiment (exp010) moves to RGRBench’s 89 packages. The pipeline receives only each package’s requirements document (user-story acceptance crite-

²gpt-4o-mini is OpenAI’s GPT-4o mini. The verifiers are OpenAI’s gpt-5.6-terra, Google’s gemini-2.5-pro (via Vertex AI), and Anthropic’s claude-sonnet-4-5. All models are called through a LiteLLM shim.

Table 2: Configuration for the primary role comparisons and RGRBench anchor; exp013 varies the judge threshold. Generation and judge temperatures are requested explicitly. In the single-function runners, late generation attempts can use the same model at temperature 1.0. Provider handling of these requests can differ. Adapter generation has its own output-token limit.

Setting	Value
Code-generation temperature	0.7
Test-generation temperature	0.7 (weak author) / 1.0 (frontier author)
Judge temperature	1.0
Judge acceptance threshold τ	0.70
Retries R (RED and GREEN)	3
Test-execution timeout	30 s
Oracle timeout	harness-specific (LCB parameter: 6 s; RGRBench: 120 s)

ria derived solely from the tests, with the tests themselves never shown) and is graded by the hidden, hand-verified suite through a non-cheating adapter (Sec. 5.4). The design keeps the judge-role contrast: a *control* cell (weak model in every role) versus a *strong-judge* cell (GPT-5.6 Terra in all judge duties), with $N=3$ runs per package per cell (534 runs). Since RGRBench labels each package by difficulty tier and the harness records whether a claim was made on the first try or only after retries, these runs describe associations with complexity (H-D4) and additional attempts (H-D2). Neither difficulty tier nor retry count is randomized.

Statistical protocol and reporting correction. For oracle-pass and false-accept contrasts in the single-function experiments, the original analyzers take each problem’s majority outcome over five runs and apply exact McNemar tests, alongside practical thresholds on mean per-run differences. We call these combined requirements the *decision bar*. Majority outcomes and mean changes are different estimands. Failing this bar does not establish absence of an effect. The original scripts omit paired tests for false rejects and total verdict error, and do not apply the protocols’ larger-control-spread adjustment; these are analysis deviations. Reported p -values are nominal, without multiplicity correction.

The original HumanEval summaries combine base-oracle pass and false-reject counts with HumanEval+ false accepts. Table 3 corrects this to use HumanEval+ throughout, recomputed from retained scores. The original summaries remain available. A separately labeled post hoc audit provides complete matrices for both oracles, the missing paired tests, and problem/package-cluster bootstrap intervals that retain repeated runs together (experiments/review_audit/). These supplemental analyses do not retroactively become pre-registered confirmations.

Telemetry. Retained successful LLM calls are logged with token counts, cost, and a problem, cell, or run tag. This telemetry enables the mechanism analysis described in Sec. 5.3, and it is included with the artifact. Table 2 lists the configuration that remains fixed across all cells.

Threats addressed by design. HumanEval is the field’s best-known benchmark and predates the training data of all current models, so we consider it within-distribution. The LiveCodeBench window serves as the off-distribution check, since its problems were released more than a year after the weak model’s training data. In the audited primary runs, we checked all 7,962 logged prompts from both experiments and confirmed that rendered benchmark identity is conveyed through the statement and worked examples, without a benchmark-name or problem-number label; generated tests, code, and feedback also appear in prompts. A date after the generator’s reported cutoff reduces contamination concerns but does not prove absence of exposure, particularly for later verifiers (Sec. 7). Both problem sets are selected for measurability (with increased hardness; within the weak-model band), so per-run counts do not represent benchmark rates and are never reported as such.

Table 3: Complete result matrix: per-run means over $N=5$ runs of 30 problems (oracle-consistent HumanEval+ values from the post hoc reporting correction). FA = false accepts (HumanEval: strong oracle; LCB: hidden oracle); FR = false rejects. The implemented decision bar uses a fixed minimum effect size (pass 4, FA 2, FR 2) and exact McNemar $p < .05$; † = past that fixed effect-size threshold, * = clears the full bar.

Benchmark	Cell	Oracle pass	FA	FR	tests/run
HumanEval subset-30 (strong oracle)	control (W everywhere)	22.6	4.0	5.0	84
	S authors tests	24.2	3.2	3.6	89
	S judges	23.4	4.4	2.2 †	105
	S authors + judges	24.8	3.4	1.6 †	101
LiveCodeBench lcb30 (hidden oracle, post-cutoff)	control (W everywhere)	15.2	4.8	3.4	76
	S authors tests	16.4	2.8	2.2	140
	S judges	20.2*	2.4 †	0.0 †	182
	S authors + judges	21.4*	3.0	0.0 †	194

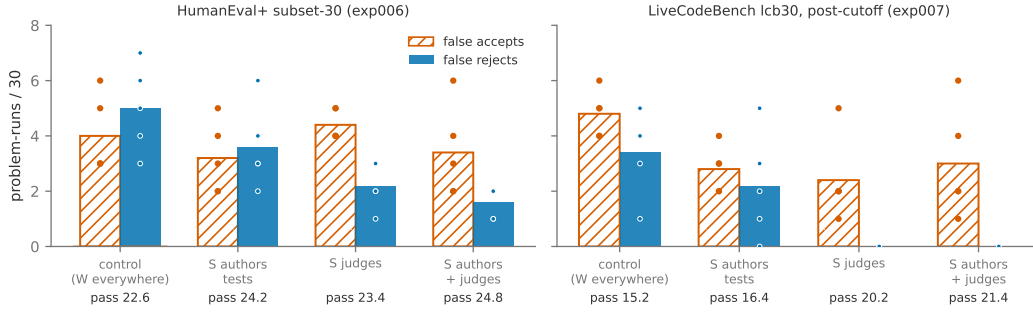


Figure 2: False accepts (hatched) and false rejects (solid) per cell; bars are means, dots the five runs; oracle pass means beneath each cell. The asymmetry is the finding: false accepts remain in every cell, while the primary strong-judge cells reduce observed false rejects and (right panel) increase oracle passes. HumanEval uses the corrected plus-oracle matrix.

5 Results

5.1 RQ1: the gap on the corrected instrument

Table 3 presents the full result matrix; Figure 2 shows the error breakdown. On the all-weak control, the gap appears on both benchmarks and both sides. For each 30-problem run, HumanEval+ has 22.6 oracle passes, 4.0 false accepts, and 5.0 false rejects in control. LiveCodeBench has 15.2 oracle passes, 4.8 false accepts (29% of claims), and 3.4 false rejects. The LCB problem dates are after the generator’s reported cutoff, and private tests are not included in prompts. These controls reduce exposure concerns without proving that every problem is novel to every model. Both error labels are relative to the chosen oracle and the pipeline’s complete set of acceptance gates.

5.2 RQ2: frontier models in the verification roles

No primary false-accept contrast meets the decision bar. On HumanEval, mean false-accept counts change from 4.0 in control to 3.2/4.4/3.4 under a strong test author, judge, or both (all nominal $p=1.0$). On LiveCodeBench, they decrease from 4.8 to 2.8/2.4/3.0, with nominal $p \in [0.125, 0.375]$. These estimates include potentially useful reductions, especially for the strong judge. The primary tests do not support the specified positive claim; equally, their null results do not establish invariance. False accepts persist in every measured condition.

Residual errors under two more frontier verifiers. Exp011 repeats the design with Gemini-2.5-Pro and Claude-4.5-Sonnet. CodeJudgeBench reports strong results for Gemini-2.5-Pro and the earlier Claude-4-Sonnet, but did not evaluate the Sonnet 4.5 model used here [11]. No false-accept contrast meets the specified bar (Table 4); several point estimates decline. False-reject changes are

Table 4: False accepts per 30-problem run (mean, $N=5$) under three frontier verifiers, each swapped into the test-author (S_{tests}), judge (S_{judge}), or both roles against the fixed weak generator. For every verifier, on both benchmarks, no strong-cell contrast clears the decision bar. This is not an equivalence test against the all-weak control. GPT-5.6 Terra is exp006/exp007; Gemini-2.5-Pro and Claude-4.5-Sonnet are the exp011 replication, with collection deviations disclosed (Sec. 5.2). HumanEval is scored with HumanEval+.

Frontier verifier S	control	S tests	S judge	S both
<i>HumanEval+</i>				
GPT-5.6 Terra	4.0	3.2	4.4	3.4
Gemini-2.5-Pro	4.8	3.2	4.0	4.8
Claude-4.5-Sonnet	3.8	2.8	3.4	4.2
<i>LiveCodeBench</i> (hidden, post-cutoff oracle)				
GPT-5.6 Terra	4.8	2.8	2.4	3.0
Gemini-2.5-Pro	3.8	1.8	3.4	2.6
Claude-4.5-Sonnet	3.6	2.0	3.4	3.6

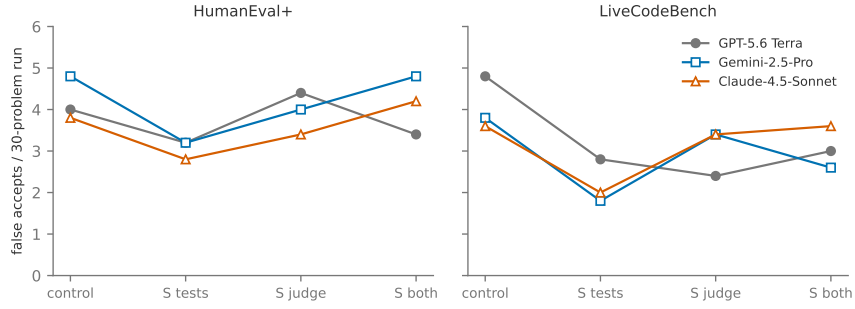


Figure 3: Mean false accepts across role assignments for each verifier (same data as Table 4). Several estimates decrease, but no contrast meets the original decision bar. Residual false accepts remain in every condition.

not uniform across providers: a strong judge does not eliminate them in every replication. Collection-time validity and rerun changes are documented in exp011’s deviation log and must accompany interpretation of these retained runs.

The primary frontier judge reduces observed false rejects. On LiveCodeBench, both primary frontier-judge cells have zero false rejects in all ten runs (control: 3.4). On HumanEval+, false rejects fall from 5.0 to 2.2 (judge) and 1.6 (both). These are descriptive reductions; the completed majority-outcome tests do not establish a false-reject effect under the full decision rule.

The primary frontier judge increases LiveCodeBench oracle passes. Oracle passes rise from 15.2 to 20.2 (judge; $\Delta+5.0$, nominal $p=.0312$) and 21.4 (both; $\Delta+6.2$, nominal $p=.0156$), meeting the original decision bar. Neither corresponding HumanEval+ contrast meets that bar. These nominal tests are not adjusted for the multiple comparisons. The post hoc audit also computes tests omitted by the original analyzer, so these should not be described as the only detectable outcomes.

No pass-rate benefit is established for a frontier test author. The strong-test-author cell has higher mean oracle passes than control on both benchmarks, but neither primary comparison meets the decision bar. FA and FR point estimates also decrease. The result is a failure to establish the specified benefit, not proof that test authorship quality has no effect.

5.3 RQ3: telemetry is consistent with a feedback benefit

Canonical tests average 75 per run in the LCB cells. Total test volume changes from about 76 in control to 182–194 in the frontier-judge cells, while cycle completions rise from about 17/30 to

An omitted exception type → a requirements–oracle mismatch

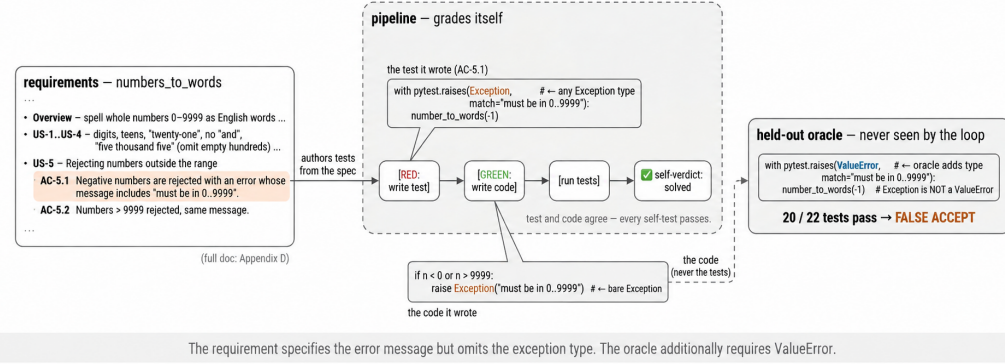


Figure 4: A requirements–oracle mismatch (RGRBench’s `numbers_to_words`). AC-5.1 specifies the error message but leaves the exception type unspecified. The self-test accepts any `Exception`, which the code raises; the oracle additionally requires `ValueError`. The resulting false accept is relative to the oracle, not demonstrated violation of the visible text. Code snippets are simplified; full excerpts appear in App. D.

23–24/30. A frontier test author also produces larger suites (140 tests per run), without a pass-rate contrast meeting the decision bar.

These observations are consistent with stronger feedback helping generation. They do not identify feedback as the cause: the judge intervention jointly changes scores, directives, retry paths, test volume, and the selected outputs. The strong-test-author condition also retains iterative review, so it is not a one-shot comparator. A feedback-only versus verdict-only ablation on matched candidates would be needed to separate these mechanisms.

5.4 RQ4: oracle disagreement at application scale

In the RGRBench control, 65 of 261 valid runs end in a claimed success, of which 41.5% fail the oracle. The strong-judge cell claims 50 successes among 251 valid runs, with a conditional gap of 42.0% (Table 5). The planned 267 runs per cell reduce to these valid totals because evaluations without usable oracle verdicts are excluded: 21 exclusions follow adapter import failures at test collection, and one control run has a harness environment error. A post hoc missing-verdict sensitivity analysis is included with the artifact. It does not resolve possible oracle or adapter errors among valid cases.

The table preserves nominal run-level Wilson intervals. These treat repeated package-runs as independent and understate uncertainty when their errors cluster. A paired bootstrap that resamples whole packages gives a strong-minus-control conditional-gap difference of +0.5 percentage points, with interval $[-19.4, 20.8]$. This allows substantial improvement and harm; the data do not establish equivalence. Cross-benchmark point estimates are descriptive because tasks, oracle contracts, and adapters differ.

RGRBench grades candidates through a generated adapter constrained to expose their APIs under the oracle’s import names. The selected calibration fixtures have no observed adapter failures, but this is not a zero error bound (Sec. 7). The example below also demonstrates that oracle failure need not mean violation of the supplied requirement.

Higher observed gap rates on harder packages. Conditional false accepts rise across difficulty tiers (Figure 5): 31% → 54% → 86% in control, and 38% → 50% → 75% under the strong judge. The hardest control tier has only seven claimed successes, six rejected by the oracle. These small, selected denominators and nonrandom difficulty assignments limit any causal or general scaling interpretation.

Table 5: RGRBench (application scale): the self-verification confusion matrix and the false-accept analyses, over $N=3$ runs of 89 packages (534 runs). Control = weak model everywhere; strong-judge = frontier model in all judge roles. TA/FA/FR/TR are counts over valid runs (accept/reject $\times$ oracle pass/fail); oracle-passing solutions = TA+FR. Conditional FA is FA as a fraction of claimed successes (TA+FA); tier and attempt rows restrict that quantity to each subset. Brackets are nominal 95% run-level Wilson intervals; package-cluster uncertainty is reported in the text.

	control	strong-judge
valid runs	261	251
true accepts (TA)	38	29
false accepts (FA)	27	21
false rejects (FR, H-D3)	14	0
true rejects (TR)	182	201
oracle-passing solutions (TA+FR)	52	29
claimed successes (TA+FA)	65	50
conditional FA (H-D1)	0.415 [0.304, 0.537]	0.420 [0.294, 0.558]
beginner tier (H-D4)	0.31	0.38
intermediate tier	0.54	0.50
advanced tier	0.86	0.75
first- / multi-attempt FA (H-D2)	0.35 / 0.64	0.34 / 0.67

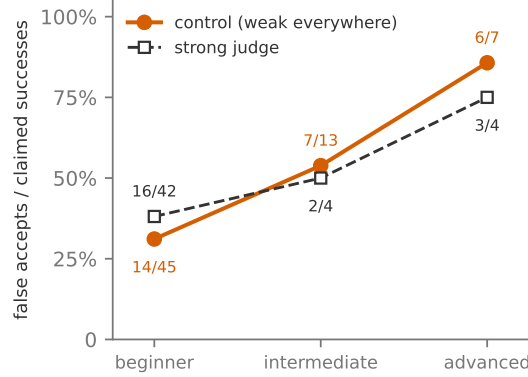


Figure 5: Conditional false accepts rise monotonically with difficulty tier on RGRBench, in both cells (H-D4). The strata are observational, and the advanced tier has few accepted runs.

Additional attempts are associated with higher gap rates. Claims reached after multiple GREEN attempts have higher conditional false accepts than first-attempt claims: 64% versus 35% in control, and 67% versus 34% with the strong judge. Retry count is a consequence of earlier failures; difficult cases are more likely to retry. The design therefore cannot distinguish selection from a harmful effect of refinement, and it does not provide a causal replication of the refinement intervention in Ahmed et al. [1].

Acceptance and correct output must be considered together. Although the conditional gap estimates are close (41.5% versus 42.0%), the strong judge reduces oracle-passing outputs from 52 to 29 and true accepts from 38 to 29. It rejects the self-authored suite before code generation on 189 of 251 valid runs; only 62 reach GREEN, compared with 258 under the weak judge. Zero false rejects consequently does not imply improved recovery of correct code. The intervention changes which implementations exist and which are accepted, so this is a comparison of pipeline policies, not judge accuracy on fixed candidates.

Threshold follow-up: different operating points, incomplete matching. Exp013 varies the threshold shared by the RED and GREEN judges. The original new points are $\tau \in \{0.40, 0.55, 0.85\}$, with $N=1$ per package; 0.70 reuses exp010. A later $\tau=0.50$ follow-up uses

Table 6: RED/GREEN threshold sweep on the strong-judge RGRBench cell (exp013). The adaptive $\tau=0.50$ follow-up has $N=3$; other new points have $N=1$; $\tau=0.70$ reuses exp010 ($N=3$). Acceptance is not matched to control. Brackets are nominal 95% run-level Wilson intervals.

cell	accept%	FA% [95%]	RED-gate%
control (weak), $\tau=0.70$	24.9	41.5 [30.4, 53.7]	1.1
strong, $\tau=0.85$	7.4	33.3 [9.7, 70.0]	91.4
strong, $\tau=0.70$	19.9	42.0 [29.4, 55.8]	75.3
strong, $\tau=0.55$	20.0	52.9 [31.0, 73.8]	72.9
strong, $\tau=0.50$ ($N=3$)	31.5	47.5 [36.9, 58.3]	62.6
strong, $\tau=0.40$	43.0	67.6 [51.5, 80.4]	47.7

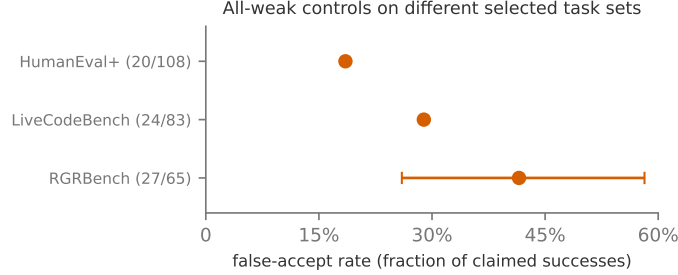


Figure 6: Oracle-relative gap rates in the primary all-weak controls. The benchmarks have different tasks and oracle contracts; this is not a shared population estimate or a causal scale comparison. The RGRBench interval resamples packages with their repeated runs. Related SWE-bench studies also report errors after test acceptance, using distinct acceptance events and denominators [1, 20].

$N=3$ (Table 6). That point is outside the original matching candidate set, and no separately frozen selection amendment is present; we treat it as adaptive follow-up rather than confirmatory matching.

Acceptance at 0.50 is 31.5%, versus 24.9% in control, so throughput was not equalized. Its conditional gap is 47.5%, versus 41.5% in control. A post hoc paired package bootstrap gives a difference of +6.0 percentage points with interval $[-9.4, 21.5]$, supporting neither superiority nor equivalence. At 0.40, both acceptance and the observed gap are higher; at 0.85, acceptance is low and the gap estimate has wide uncertainty. The rates are not monotonic at every intermediate point. The sweep shows sensitivity to the operating threshold, but a weak-judge threshold curve and matched candidate evaluation would be needed to compare discrimination at similar throughput.

6 Discussion

Persistent errors do not establish invariance. Every tested configuration retains oracle-relative false accepts. None of the original false-accept contrasts meets the registered joint decision rule, but this does not establish equivalence or an inability of stronger models to help. Several LiveCodeBench estimates favor stronger verification roles. Counts of false accepts and the fraction of accepted claims that are false also answer different questions: changing the number and composition of accepted outputs can change the latter without a corresponding count reduction. The result is evidence of residual error under these policies, with uncertainty about its reduction, rather than a general law about verifier strength.

Feedback and selection change together. Larger suites, more completed cycles, and higher oracle pass rates in the primary LiveCodeBench judge cells are consistent with useful reviewer feedback. However, assigning a stronger judge changes test acceptance, code review, failure analysis, and subsequent candidates simultaneously. These experiments do not isolate feedback as the causal source of the gains, nor separate improved generation from improved selection. The effect also varies across settings: the RGRBench judge rejects many packages at RED and reduces correct-output yield, while the Gemini judge-only LiveCodeBench replication does not reproduce the primary experiment’s false-reject reduction. Strong test authors can retry RED too, so their results do

not compare a one-shot author with an iterative judge. Costs and outcomes must be compared by configuration before recommending an efficient deployment.

The review contract matters. The GREEN judge is instructed to evaluate code against the generated tests, and receives the task title rather than the full original specification. The RED judge does see that specification, but uses a scoring rubric that can accept a suite with a deducted error at the chosen threshold. Consequently, the experiment evaluates stronger models under a particular test-centered review contract. A passing suite supplies no executed counterexample to its own omissions, but a reviewer given the original specification could still identify them. It is not logically impossible for an internal reviewer to reduce false accepts. This distinction connects the results to the established oracle problem [2] and limits of resampling under imperfect verification [18].

A direct missing baseline. A useful next experiment freezes the final candidates and their original self-verdicts, then asks a separate reviewer to compare each implementation with the full original specification. Its accept/reject predictions can be evaluated against the same held-out oracle without changing the candidates. Comparing specification-aware review with the current test-centered GREEN rubric would test whether the prompt’s information and objective limit gatekeeping. A separate randomized ablation that withholds reviewer feedback while preserving verdicts would help isolate coaching. Both should report conditional error, acceptance, correct-output yield, and cost; neither baseline has been run here.

Specifications and oracles can disagree. Passing tests is not a proof of conformance to every requirement, and failing held-out tests is not always evidence of violating the supplied text. The `numbers_to_words` case exposes this second possibility: the requirements leave the exception type unspecified while the oracle requires `ValueError` (App. D). This is a specification-oracle contract mismatch, not an established implementation error. The distinction between verification and validation [4] helps frame the problem, but does not identify when every observed disagreement originated. Clarifying acceptance criteria with human input, as in TiCoder [6], is a plausible intervention. Its effect on this pipeline remains to be measured. Blinded adjudication of candidate failures against both requirements and oracle expectations would establish how much of the reported gap reflects semantic implementation defects.

Implication for builders. Treat self-test success as an imperfect signal whose reliability must be measured for the intended task distribution and acceptance policy. Held-out tests, independently checked specifications, and differential or metamorphic checks can provide additional evidence, each with its own limitations. The current results justify checking accepted outputs; they do not show that self-verdicts contain no information, that all stronger reviewers fail, or that an external test suite is infallible.

7 Threats to Validity

Construct and oracle adequacy. All correctness labels are relative to the stated benchmark oracle. Incomplete tests can miss defects, while an incorrect or underspecified oracle contract can reject behavior allowed by the prompt. HumanEval’s base and plus suites are distinct oracles; the post hoc reporting correction uses complete matrices from each rather than combining base passes and false rejects with plus false accepts. On RGRBench, the mutation kill rate characterizes detection of selected mutations, not the probability that the oracle’s expected behavior matches the requirements. Classifying surviving mutants does not provide a semantic error bound. Positive self-verdicts require passing execution; negative verdicts additionally include judge rejection, exhausted retries, or failure to produce an implementation. Thus false rejects are not exclusively cases of faulty tests vetoing otherwise correct code.

Adapter calibration. The calibration exercises model-written adapters on behavior-preserving transformations of reference implementations. It observes no adapter errors in 13 retained fixtures, after excluding 5 fixtures whose deterministic adaptation failed. These selected cases preserve much of the reference structure and do not cover arbitrary candidate APIs. The recorded calibration error is a fraction of tests failed, whereas the study’s binary endpoint fails a candidate if any oracle test fails. Zero observed calibration errors neither gives a zero population error bound nor excludes

adapter-caused disagreements in the experimental runs. Adapter and specification mismatches remain possible contributors to the application-scale measurements.

Selection and repeated packages. Model and threshold changes alter both acceptance and the candidates reaching a usable oracle verdict. Restricting a comparison to packages valid in both cells does not match accepted candidates; similar throughput would not do so either. The threshold experiment changes RED and GREEN together, and the $\tau=0.50$ follow-up is adaptive and does not exactly match control acceptance. These are comparisons of pipeline policies, not estimates of reviewer accuracy on a fixed candidate population. Nominal Wilson intervals treat package-runs as independent; repeated runs on the same package can be correlated. The supplementary package bootstrap retains repeats and pairing and leaves broad uncertainty about the conditional-error contrasts. It describes resampling of these selected packages, not representative sampling of software projects. Three new threshold points have only one run per package, limiting interpretation of the operating curve. Retry and difficulty-tier comparisons are also observational: retrying tasks can be harder before the retry occurs.

Registration and collection deviations. The authors retained protocols intended to precede collection, but the public history was reconstructed for release; historical run SHAs refer to a private working history. The released Git graph alone does not independently establish the original registration chronology. In exp011, partial runs were deleted and repeated after provider outages, using a nonempty-implementation inclusion threshold that was subsequently relaxed. This differs from the protocol’s treatment of terminal API errors as failed runs and can also select on legitimate pipeline abandonment. Without the discarded attempts, the resulting selection and cost effects cannot be fully reconstructed. Cross-provider findings therefore refer to retained runs, with this additional limitation.

Conclusion validity. The original tests dichotomize each problem’s repeated outcomes by majority, while effect floors use mean per-run counts; these are different estimands. Some registered false-reject and total-error tests and the control-spread adjustment were omitted by the original runners. The review supplies completed tests and clustered intervals as explicitly post hoc analyses, with nominal, unadjusted p -values. They must not be relabeled as prospective confirmations. Neither failure of a significance test nor failure to exceed a practical floor establishes an effect of zero. With small selected task sets, repeated outcomes, and numerous contrasts, meaningful benefits and harms remain compatible with several comparisons.

Scope and contamination. The main interventions keep one implementation-generator model fixed in one pipeline. All three verifiers are evaluated on the two single-function subsets; only GPT-5.6 Terra is evaluated on RGRBench. Model aliases and available historical metadata do not fully identify immutable served snapshots. The LiveCodeBench window postdates the weak model’s documented cutoff and reduces a known exposure risk, but does not establish that task ideas or variants were unseen. Later verifiers may have encountered the tasks; exposure could change tests, acceptance, feedback, and candidate selection, so its effect on conditional error has no established direction. Differences between benchmarks also combine changes in problem selection, specifications, and scoring contracts with application scale. The results should not be treated as benchmark-wide rates or a general complexity law.

Artifact and execution limits. The release supports reanalysis of stored outcomes, with incomplete historical provider snapshots, request settings, and some execution traces; exact API replay is not established. Logged costs estimate retained successful calls and can omit discarded or failed attempts; offline adapter grading also contributes to study spend. Generated code runs in temporary directories under subprocess timeouts, not a hardened sandbox with network, memory, and filesystem isolation. Timeout policies differ between self-testing and the benchmark harnesses. Fresh replication should record these settings and use appropriate execution isolation.

Anchoring. The artifact includes canonical-example anchoring variants, but these do not provide a false-accept comparison sufficient to establish that anchoring cannot help. The case studies show that covering worked examples can coexist with a held-out failure. A fully instrumented anchoring ablation with repeated runs remains a separate experiment.

8 Conclusion

A self-testing pipeline can accept implementations that fail held-out tests. We measure that disagreement across selected single-function problems and RGRBench packages, and show why acceptance, oracle success, and abandonment must be reported together. Frontier verification leaves residual false accepts in every measured condition, while some interventions improve oracle success and several false-accept estimates decrease. The experiments do not establish that verifier strength cannot help or that error rates are invariant. At application scale, stricter gating reduces correct output, and requirements, oracles, and adapters add measurement uncertainty.

For builders, passing self-authored tests is useful evidence with an empirically measurable error rate, not a correctness guarantee. Independent evaluation and explicit verdict accounting remain necessary to assess that evidence. The released instrument and post hoc reporting corrections provide a basis for stronger tests of specification-aware review, feedback mechanisms, and oracle fidelity across more pipelines and generators.

References

- [1] Toufique Ahmed, Jatin Ganhotra, Avraham Shinnar, and Martin Hirzel. Investigating test overfitting on SWE-bench. *arXiv preprint arXiv:2511.16858*, 2025.
- [2] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015. doi: 10.1109/TSE.2014.2372785. URL <https://coinse.github.io/publications/pdfs/Barr2015qd.pdf>.
- [3] Kent Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2002.
- [4] Barry W. Boehm. Verifying and validating software requirements and design specifications. *IEEE Software*, 1(1):75–88, 1984.
- [5] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. Codet: Code generation with generated tests. In *ICLR*, 2023.
- [6] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K. Lahiri. Llm-based test-driven interactive code generation: User study and empirical evaluation. *IEEE Transactions on Software Engineering*, 2024. ICSE 2025 journal-first.
- [7] Kevin Han, Siddharth Maddikayala, Tim Knappe, Om Patel, Austen Liao, and Amir Barati Farimani. TDFlow: Agentic workflows for test driven development. *arXiv preprint arXiv:2510.23761*, 2025.
- [8] Sirui Hong et al. Metagpt: Meta programming for a multi-agent collaborative framework. *ICLR*, 2024. arXiv:2308.00352.
- [9] Yiran Hu, Shanchao Liang, Nan Jiang, Yi Wu, and Lin Tan. TENET: One step toward test-driven development for repository-level code generation. *arXiv preprint arXiv:2509.24148*, 2025.
- [10] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- [11] Hongchao Jiang, Yiming Chen, Yushi Cao, Hung-yi Lee, and Robby T. Tan. Codejudgebench: Benchmarking llm-as-a-judge for coding tasks. *arXiv preprint arXiv:2507.10535*, 2025.
- [12] Haolin Jin and Huaming Chen. Uncovering systematic failures of LLMs in verifying code against natural language specifications. *arXiv preprint arXiv:2508.12358*, 2025.
- [13] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatGPT really correct? rigorous evaluation of large language models for code generation. In *NeurIPS*, 2023. arXiv:2305.01210.

- [14] Noble Saji Mathews and Meiyappan Nagappan. Test-driven development for code generation. *arXiv preprint arXiv:2402.13521*, 2024.
- [15] Fabio Pereira. TTDD: Tautological test-driven development (anti-pattern). <https://fabiopereira.me/blog/2010/05/27/ttdd-tautological-test-driven-development-anti-pattern/>, 2010.
- [16] Jason Rudolph. Testing anti-patterns: The ugly mirror. <https://jasonrudolph.com/blog/2008/07/30/testing-anti-patterns-the-ugly-mirror/>, 2008. Blog post; catalogued in the Test Smells Catalog.
- [17] Brian Sam-Bodden. What context does a coding agent actually need to act? *arXiv preprint arXiv:2607.09691*, 2026. doi: 10.48550/arXiv.2607.09691. URL <https://arxiv.org/abs/2607.09691>.
- [18] Benedikt Stroebel, Sayash Kapoor, and Arvind Narayanan. The limits of inference scaling through resampling. *arXiv preprint arXiv:2411.17501*, 2026. doi: 10.48550/arXiv.2411.17501. URL <https://arxiv.org/abs/2411.17501v3>. Version 3; first circulated in 2024 as “Inference Scaling fLaws: The Limits of LLM Resampling with Imperfect Verifiers”.
- [19] Hamed Taherkhani, Alireza DaghighFarsoodeh, Mohammad Chowdhury, Hung Viet Pham, and Hadi Hemmati. Consistency meets verification: Enhancing test generation quality in large language models without ground-truth solutions. *arXiv preprint arXiv:2602.10522*, 2026.
- [20] Boxi Yu, Yang Cao, Yuzhong Zhang, Liting Lin, Junjielong Xu, Zhiqing Zhong, Qinghua Xu, Guancheng Wang, Jialun Cao, Shing-Chi Cheung, Pinjia He, and Lionel Briand. SWE-ABS: Adversarial benchmark strengthening exposes inflated success rates on test-based benchmark. *arXiv preprint arXiv:2603.00520*, 2026.

A Defect register of the studied pipeline (summary)

A line-by-line review of the pipeline surfaced 15 harness defects (H-1–H-15: import-time side effects, environment-dependent tool invocation, missing response logging, list-content crashes on 2026 models, usage-logger stacking) and nine behavioral defects (Table 7). All harness defects and behavioral defects B-1–B-8 are fixed in the study instrument; B-10’s underlying mechanism was later removed outright, so the failure analyst no longer produces code at all and nothing the judge writes can ship. The prompt templates were separately rewritten after review (Sec. 3.3). Identifiers are stable against the repository’s full register, which includes locations, failure scenarios, and dispositions; that register never assigned the identifier B-9, so the nine behavioral defects carry the identifiers B-1–B-8 and B-10, and Table 7 has no B-9 row by construction.

Table 7: Behavioral defects found during review; disposition in the study instrument.

ID	Finding	Disposition
B-1	Template render errors silently become empty prompts upstream	fixed
B-2	Hanging generated test aborts the whole problem run	fixed
B-3	Judge pass/fail verdicts via substring scan of the entire response	fixed
B-4	First-match score regex; “Score: 85” normalizes to a perfect 1.0	fixed
B-5	GREEN judge verdict threshold hardcoded at 0.7, ignoring configuration	fixed
B-6	Two result classes with different validation semantics across judges	fixed
B-7	Latent <code>NameError</code> in the RED retry fallback (unreachable)	fixed
B-8	GREEN’s execution gate tests one file; VERIFY tests the whole suite	fixed
B-10	Judge-corrected code never ships; travels only as prompt text	mechanism removed

B Prompt skeletons

Abbreviated structure of the two RED-phase templates (the GREEN pair follows the same pattern; full templates in the artifact). Anchoring blocks are marked in the source and removed mechanically to create ablation variants.

RED generator (test author).

You are writing the test suite for a Python function BEFORE any implementation exists (TDD, RED phase). Your tests are the only specification the implementer will see [...]

```
## Task          <- title, description, signatures
## Specification details  <- full problem statement
## Scenarios to cover    <- worked examples from the spec
[ANCHOR] ## MANDATORY: Canonical Examples from Specification
  For EVERY example: one test named test_canonical_example_N with
  the EXACT inputs and EXACT expected output + derivation comment.
## Rules  1. only specified behavior ... 5. plain pytest
## Reviewer feedback on your previous suite  <- retry only
## Output: one fenced ‘python block, nothing else
```

RED judge (test reviewer).

Judge one thing only: does this suite faithfully verify the specification? [...]

```
[ANCHOR] ## Check 0 - MANDATORY Canonical Example Coverage
  any missing/altered example => Final score: 0.00
## Checks  1. fidelity (recompute every expected value)
           2. no invented constraints  3. coverage  4. sanity
## Scoring: deduction schedule (0.30/0.30/0.20/0.10, floor 0)
## Output: findings; ADD:/REMOVE:/FIX: directives (apply-verbatim);
           last line exactly 'Final score: <x.xx>'
```

C False-accept case studies

The following HumanEval examples share a pattern: the self-authored suite anchors itself to the specification’s worked examples (the anchoring block of App. B enforces one `test_canonical_example` per example), the implementation passes these, and both miss a behavior that the held-out oracle checks. Three HumanEval false accepts make this pattern specific: two from the all-weak control and one from the cell with a frontier judge, each accepted with a top GREEN-judge score. Code and tests are copied exactly from the committed run logs.

HumanEval/32 (find_zero, control cell). The spec requests a root of a polynomial with an even number of coefficients (hence odd degree) under the stated leading-coefficient assumption. The self-suite tests only the two docstring examples (`find_zero([1,2]) → -0.5`, `find_zero([-6,11,-6,1]) → 1.0`). The implementation uses Newton’s method starting from `x=0.0` for 100 iterations. Newton-from-zero converges on both example polynomials, so the suite passes, but this is not a general root-finder: when the iteration stalls (`dfx==0`, and it breaks) or diverges, it returns a non-root. HumanEval+’s extra polynomials cause exactly this, and the oracle rejects. The tests only confirm the two points the code was tailored for, not the full specification.

HumanEval/74 (total_match, control cell). The spec returns the list with the smaller total character count, *and returns the first list if the totals are equal*. The implementation handles the tie incorrectly: `if total1 < total2: return lst1 else return lst2`, so equal totals default to `lst2`. None of the five generated example tests presents a clear tie—the only equal-total case is `([], [])`, where both answers are `[]`, so the tie branch is never tested. HumanEval+ includes an equal-total pair with different contents; the code returns the second list where the spec requires the first, and the oracle rejects.

HumanEval/153 (Strongest_Extension, frontier-judge cell). Strength is calculated as (uppercase count) minus (lowercase count); the strongest extension is the one with the highest value, with ties going to the earliest occurrence. The implementation sets the running maximum to 0 at the start and only updates it when `strength > strongest_strength`. This means if every extension has a strength of zero or less, and the strongest is not the first one, the loop never updates and it ends up returning `extensions[0]`. The two generated tests do not trigger this scenario—example 1 is all-negative, but its first element is already the strongest, so the bug is hidden. As a result, the suite

passes, and the frontier judge gave it a score of 1.0. HumanEval+ provides an all-negative case where the maximum is not the first, and in that case, the oracle rejects it.

At application scale. RGRBench also contains oracle-relative disagreements whose source is the requirements contract. The `strong_judge` run on `numbers_to_words` certifies itself, but the held-out oracle only passes 20 out of its 22 tests; App. D repeats this false accept completely. The per-package artifacts (including self-authored tests, the generated package, and the oracle’s result) are committed for all 534 runs.

D An RGRBench requirements–oracle mismatch: `numbers_to_words`

The false accept of Figure 4, laid out end to end. The pipeline sees only the requirements below; from them it authors its tests and its implementation, and reports success once the implementation passes those tests. The held-out oracle is applied afterward and never enters the loop. The implementation passes the oracle’s spelling tests; the observed disagreement is the exception *type* on the range check. The requirement fixes the message but not the type, the self-test accepts any `Exception`, and the oracle demands `ValueError`. This example supports an oracle-relative gap, but does not establish that the agent misread or violated its supplied requirements. The historical input is retained; silently adding the missing type now would change the evaluated task.

Requirements given to the pipeline. User-story narratives are omitted; the acceptance criteria are verbatim.

```
# Spelling numbers as English words

## Overview
A number-spelling service turns any whole number from 0 through
9999 into its English words, following everyday conventions:
hyphenated compounds for twenty-one through ninety-nine, no "and"
between parts, and plain spaces between the thousands, hundreds,
and remainder portions.

### US-1: Spelling numbers up to twenty
- AC-1.1: Zero is spelled "zero", and single digits use their
  names, such as "five" and "eight".
- AC-1.2: Ten is spelled "ten", and the teens are single
  unhyphenated words, such as "thirteen" and "nineteen".

### US-2: Spelling two-digit numbers
- AC-2.1: Round tens are single words with no hyphen, such as
  "twenty" and "ninety".
- AC-2.2: Other two-digit numbers are hyphenated tens-units
  compounds: 21 is "twenty-one", 77 is "seventy-seven", and 99
  is "ninety-nine".

### US-3: Spelling hundreds
- AC-3.1: 100 is "one hundred"; the leading "one" is always kept.
- AC-3.2: A hundreds part is followed by the spelled remainder
  separated by a space: 303 is "three hundred three", 555 is
  "five hundred fifty-five", and 115 is "one hundred fifteen".
- AC-3.3: The word "and" never joins the hundreds to the
  remainder.

### US-4: Spelling thousands
- AC-4.1: 1000 is "one thousand" with the leading "one" kept,
  and round thousands follow the same pattern, such as "two
  thousand".
- AC-4.2: The thousands part is followed by the spelled
  remainder: 2400 is "two thousand four hundred", 3466 is "three
  thousand four hundred sixty-six", and an empty hundreds part is
  simply omitted, so 5005 is "five thousand five".
- AC-4.3: The largest supported number, 9999, is "nine thousand
```

```
nine hundred ninety-nine".
```

```
### US-5: Rejecting numbers outside the range
- AC-5.1: Negative numbers are rejected with an error whose
  message includes "must be in 0..9999".
- AC-5.2: Numbers greater than 9999 are rejected with the same
  error message wording.
```

Tests the pipeline authored (range check). It also wrote a passing test for every spelling rule; the two range checks are the ones that decide this case. Each asks only for a generic Exception.

```
def test_reject_negative_numbers():
    with pytest.raises(Exception, match=r"must be in 0\.\.9999"):
        number_to_words(-1)                # AC-5.1

def test_reject_numbers_greater_than_...ninety_nine():
    with pytest.raises(Exception, match=r"must be in 0\.\.9999"):
        number_to_words(10000)             # AC-5.2
```

The generated implementation. Digit tables and the remainder-spelling branch (both correct) are omitted with ... and [...]; all other lines are verbatim. The range check raises a bare Exception.

```
def number_to_words(n):
    if n < 0 or n > 9999:
        raise Exception("must be in 0..9999")
    if n == 0:
        return "zero"
    units = ["", "one", "two", ...]        # digit tables abbreviated
    teens = ["ten", "eleven", "twelve", ...]
    tens = ["", "", "twenty", "thirty", ...]
    thousands = n // 1000
    hundreds = (n % 1000) // 100
    remainder = n % 100
    words = []
    if thousands:
        words.append(units[thousands] + " thousand")
    if hundreds:
        words.append(units[hundreds] + " hundred")
    if remainder:
        [...] # spell the 0..99 remainder, hyphenating 21..99
    return " ".join(words)
```

The held-out oracle (applied after the loop). The other twenty tests cover the spelling rules and all pass; the two range checks require ValueError, which a bare Exception is not, so running the full oracle against the implementation gives 20 of 22.

```
def test_negative_numbers_are_rejected() -> None:
    with pytest.raises(ValueError, match="must be in 0..9999"):
        number_to_words(-1)

def test_numbers_above_9999_are_rejected() -> None:
    with pytest.raises(ValueError, match="must be in 0..9999"):
        number_to_words(10000)
```